

Figure 3: Creating a new Android project

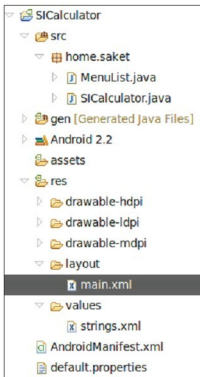


Figure 4: Project folder structure

Our fourth step is to repeat the previous step, modifying IDs and labels for the other two components, the result of which is shown in Figure 7. Our final step is to add buttons, in a manner similar to Step 3—the difference is that we add buttons instead of label text and the text-box.

Now that we're done with the UI, let's move on to the functionality of our application—to calculate simple interest based on the given input, and display it to the user.

The Java code

I have created just one Java class here; as an application grows more complex, you can have multiple Java classes performing operations.

As promised earlier, let's look at the *Activity* class. An

Activity, in Android terms, is nothing but a screen window. We extend our class with *Activity* because we need to display a screen window with the application UI. When you want to perform different actions, or have multiple screens in your application, you need to have separate Java classes extending the *Activity* class, one for each of the screen windows.

When you create a new Android project, these three lines of code will be automatically generated for you:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
}
```

The *onCreate()* method is invoked when your application starts. It calls the super-class's *onCreate()* method to set the initial parameters of the screen window. *setContentView(R.layout.main)* initialises the screen layout with the UI described in *main.xml*.

Next, your Java code has to obtain references to the UI elements, to retrieve user inputs. For this, use a method named *findViewById()*, passing the unique ID that we assigned to each UI component in *main.xml*.

```
principal = (EditText)findViewById(R.id.pAmount);
rate = (EditText)findViewById(R.id.rate);
period = (EditText)findViewById(R.id.period);
messageBox = (Button)findViewById(R.id.box);
```

You can refer to the full code listing from the GitHub repository. The basic steps taken are:

```
android:layout_marginLeft="25dp"
>
<TextView android:text="@string/principal"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/TextPrincipal"
    android:textSize="28dp"
>
</TextView>

<EditText android:id="@+id/pAmount"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:width="100dp"
    android:height="35dp"
    android:layout_marginLeft="95dp"
    android:inputType="numberDecimal"
>
</EditText>
</LinearLayout>
```



Note: 100dp is 100 density points. These units abstract away screen-size issues; they translate to differing numbers of pixels based on differing screen resolutions on different devices. That is, 100dp is different for a 7.6 cm (3 inch) screen and for a 10.2 cm (4 inch) screen, making your UI specification independent of physical screen sizes.